

QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

AI IncidentFlow – Agentic AI Incident Orchestration Platform

Document Prepared by: We-cooked-nen (Group 112)

Group Members:

Chiew Yi Jia

Tan Yi Shen

Soo Desmond

Yan Calvin

Yau Han Hong

Table OF Contents

Table OF Contents.....	2
AI IncidentFlow – Updated Testing Strategy.....	3
Document Control.....	3
1. Scope & Requirements Traceability.....	3
1.1 In Scope.....	3
1.2 Out of Scope.....	3
2. Risk Assessment & Mitigation Strategy.....	3
3. Test Environment & Execution Strategy.....	4
Unit Testing.....	4
Integration Testing.....	4
Pass Threshold.....	5
4. Test Case Specifications.....	5
TC-01 Happy Path.....	5
TC-02 Negative Test.....	5
TC-03 Escalation Test.....	5
TC-04 Agentic Tool Calling Accuracy.....	6
TC-05 Related Tickets Query Validation.....	6
TC-06 Ticket Overview & AI Analysis Retrieval Validation.....	6
5. AI Output Testing.....	6
6. Assumptions.....	7
7. AI declaration.....	7

AI IncidentFlow – Updated Testing Strategy

Document Control

Field	Detail
System Under Test	AI IncidentFlow
Objective	Validate AI ticket analysis, routing logic, tool calling accuracy, ticket creation, related tickets discovery, and escalation workflows.

1. Scope & Requirements Traceability

1.1 In Scope

- Incident submission and validation
- AI categorization, summarization, and priority assignment
- Agentic tool calling and real-time data retrieval
- Ticket creation and persistence in SQLite
- Similar incident discovery and related ticket mapping
- Escalation logic and SLA threshold management
- Ticket dashboard overview and data retrieval

1.2 Out of Scope

- Real email integration
- Notifications
- User authentication

2. Risk Assessment & Mitigation Strategy

Technical Risk	Likelihood	Severity	Risk Score	Mitigation
AI misroutes tickets	3	4	12	Utilize static Knowledge Base (Context Caching) to enforce strict enterprise routing policies. Allow manual reassignment.
AI API unavailable	2	5	10	Fallback to "Pending Manual Review" state with default values.
JSON parsing failure	1	5	5	Enforce native Structured Outputs (<code>response_format={"type": "json_object"}</code>) via the GLM API to guarantee exact schema conformity.
Escalation fails	2	4	8	Retry escalation check via automated background tasks or manual trigger.
Agentic Tool execution failure	2	3	6	Implement safe fallback mechanisms that ignore tool execution errors and rely on the original prompt description.

3. Test Environment & Execution Strategy

Unit Testing

- **Strategy Shift:** execution has pivoted from End-to-End (E2E) FastAPI testing to isolated Python unit tests (`test_ai_logic.py` and `test_db_logic.py`).
- **Validate:**
 - AI response parsing and structured output compliance
 - Agentic tool execution logic and multi-turn loops
 - Raw SQLite ticket creation and related ticket queries (using in-memory `:memory:` databases)
 - Escalation triggers

Integration Testing

- **Validate:**
 - Support form submission → AI analysis (with tool execution) → Ticket creation

© UMHackathon 2026

Email: umhackathon@um.edu.my

- Dashboard API retrieval → Ticket deserialization → UI rendering

Pass Threshold

At least 85% test pass rate, with 100% pass for critical workflow tests (ticket creation, AI output parsing, escalation).

4. Test Case Specifications

TC-01 Happy Path

- **Feature:** Incident Submission to Ticket Creation
- **Test Steps:**
 1. Submit valid incident via support form (Title, Description, Department).
 2. AI categorizes the issue and generates a structured JSON summary.
 3. Ticket is created in the database.
- **Expected Result:** Ticket is created successfully with the correct category, priority, assigned team, and default status set to 'Open'.
- **Status:** Pass

TC-02 Negative Test

- **Feature:** Empty Incident Description
- **Test Steps:**
 1. Submit incident with missing or empty description/title.
- **Expected Result:** System rejects the submission with a validation error (422 Unprocessable Entity).
- **Status:** Pass

TC-03 Escalation Test

- **Feature:** Auto Escalation Logic (Database)
- **Test Steps:**
 1. Insert a ticket with a `created_at` timestamp older than the SLA threshold (e.g., 6 minutes ago).
 2. Execute the escalation script or API endpoint.
- **Expected Result:** Ticket status changes to 'Escalated' and the assigned team is appended with '(Manager)'.
- **Status:** Pass

TC-04 Agentic Tool Calling Accuracy

- **Feature:** AI Agentic Tools
- **Test Steps:**
 1. Pass an incident report requiring external data (e.g., "The payroll server is down") to `analyze_incident()`.
- **Expected Result:** The AI successfully requests the `check_server_status` tool, the backend provides the real-time server health, and the AI incorporates this data into its final structured JSON summary.
- **Status:** Pass

TC-05 Related Tickets Query Validation

- **Feature:** Similar Incident Discovery (Database)
- **Test Steps:**
 1. Insert a new ticket into the database with a specific category.
 2. Trigger the related tickets query logic during ticket creation.
- **Expected Result:** The database lookup successfully retrieves previously opened tickets of the same category and accurately populates the new ticket's `related_tickets` JSON array, ignoring 'Closed' tickets.
- **Status:** Pass

TC-06 Ticket Overview & AI Analysis Retrieval Validation

- **Feature:** Dashboard Overview API
- **Test Steps:**
 1. Make a request to the `GET /api/tickets` endpoint.
- **Expected Result:** The API successfully parses and returns all necessary AI analysis details (summary, priority, assigned team) and related tickets for the frontend dashboard overview, avoiding any string parsing or database decoding errors.
- **Status:** Pass

5. AI Output Testing

Test ID	Prompt Input	Expected Output
AI-01	"VPN down for all staff"	<code>category</code> = network, <code>assigned_team</code> = Network Operations Center

AI-02	"Payroll inaccessible"	<code>category</code> = software, <code>assigned_team</code> = Enterprise App Support
AI-03	"Need password reset"	<code>priority</code> = medium, <code>assigned_team</code> = Identity & Access Management
AI-04	"Blue screen of death"	<code>category</code> = hardware, <code>priority</code> = medium, <code>assigned_team</code> = Hardware Field Services
AI-05	"Executive cannot login"	<code>priority</code> = critical (per Knowledge Base rules)
AI-06	"Phishing email received"	<code>priority</code> = high, <code>assigned_team</code> = Cybersecurity Incident Response

Test Case passed for all

6. Assumptions

- GLM API consistently returns valid JSON by enforcing the `response_format` constraint.
- Enterprise departments and routing policies are predefined in `knowledge_base.txt`.
- Tickets have fixed escalation thresholds (5 minutes for MVP demonstration purposes).
- Agentic tools (`tools.py`) return determinable responses for the AI to reason upon.

7. AI declaration

This document was authored solely by Team We-cooked-nen (Group 112) All core ideas, arguments, and conclusions were developed independently by the team. Generative AI was used strictly for structural refinement, formatting, and enhancing linguistic clarity. The final content remains the original intellectual product of the authors, who take full responsibility for its accuracy and integrity.